

OS_assignment_10831_2021026108

컴퓨터소프트웨어학부 2021026108 김민제

notion을 이용하여 작성한 리포트입니다.

Design

sysproc.c는 프로세스 관련 system call을 구현하는 파일이므로 sysproc.c에 getppid() 함수를 구현하여 ppid를 리턴받고 유저가 ppid를 입력했을 때 유저프로그램이 system call을 하여 받은 pid와 ppid를 출력해주는 형태로 구현을 계획하였습니다. 이미 sysproc.c에 getpid() 함수가 구현되어 있으므로 이를 참고하여 비슷하게 getppid를 비슷하게 구현하면 될 것이라 생각했습니다. 구현과정은 lab03의 실습 방식과 유사하게 진행하였습니다.

Implementation

xv6에서 getppid() 시스템 콜 구현 방법

xv6에 부모 프로세스의 PID를 반환하는 `getppid()` 시스템 콜을 추가했습니다.

1. 시스템 콜 구현하기

`getppid_syscall.c` 를 새롭게 만드는 대신 `sysproc.c`를 보강하여 만들었습니다.

2. `Makefile` 에 source file 추가

파일 유지: `Makefile`

```
OBJS = \  
.....  
$K/virtio_disk.o \  
$K/prac_syscall.o
```

기존에 존재하는 `sysproc.c`에 추가할 것이므로 `makefile`의 커널 부분은 그대로 두었습니다.

3. `defs.h` 에 함수 선언

파일 유지: `defs.h`

`getppid()`는 다른 kernel의 c 파일에서 사용되어야 할 이유가 없어 `defs.h`를 그대로 두었습니다.

3. `sysproc.c` 에 `sys_getppid()` 함수 추가 (wrapper function 완성)

파일 수정: `kernel/sysproc.c`

```
// return parent's pid ASSIGNMENT #1  
uint64  
sys_getppid(void)  
{
```

```
return myproc()→parent→pid;
}
```

myproc()→parent→pid를 사용하여 ppid를 반환하는 wrapper function을 만들었습니다.

4. `syscall.c` 와 `sys.call.h` 에 시스템 콜 테이블 등록

파일 수정: `kernel/syscall.h`

```
.....
#define SYS_myfunction 22
#define SYS_getppid 23
```

기존에 존재하는 시스템 콜 번호 다음으로 SYS_getppid를 추가했습니다.

파일 수정: `kernel/syscall.c`

```
extern uint64 sys_close(void);
extern uint64 sys_myfunction(void);
extern uint64 sys_getppid(void);

static uint64 (*syscalls[])(void) = {
    .....
    [SYS_close] sys_close,
    [SYS_myfunction] sys_myfunction,
    [SYS_getppid] sys_getppid,
};
```

시스템 콜 테이블에 sys_getppid()를 추가해 커널이 호출을 처리할 수 있게 했습니다.

user 프로그램 구현

1. `user/user.h` 에 함수 선언

파일 수정: `user/user.h`

```
//system calls
.....
int myfunction(char*); //example for lab03
int getppid(void); // declare getppid for user program
```

유저 프로그램이 getppid 시스템 콜을 사용할 수 있도록 선언해주었습니다.

5. `usys.pl` 에 새로운 매크로 추가

파일 수정: `user/usys.pl`

```
entry("myfunction");
entry("getppid");
```

usys.pl에서 시스템 콜을 호출할 수 있도록 매크로를 추가했습니다.

6. `user/ppid.c` 사용자가 실행하는 프로그램 완성

새로운 파일 생성: `user/getppid_test.c`

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[]){
    fprintf(1, "My student ID is 2021026108\n");
    uint64 pid = getpid();
    uint64 ppid = getppid();
    fprintf(1, "My pid is %ld\n", pid);
    fprintf(1, "My ppid is %ld\n", ppid);
    return 0;
}
```

학번에 대한 정보를 출력해주고 유저 프로그램에서 `getpid()`와 `getppid()`를 호출하여 현재 프로세스의 pid와 ppid를 받고 이를 형식에 맞추어 출력해주도록 했습니다.

6. `Makefile` 에 유저 프로그램 코드 파일 추가

파일 수정: `Makefile`

```
UPROGS=\
....
    $U/_my_userapp \
    $U/_ppid \
```

Makefile에 ppid라는 유저 프로그램을 추가했습니다.

Results

1. xv6 빌드 및 실행

Makefile을 통해서 xv6를 다시 빌드하고 실행합니다.

```
$ make clean && make qemu
```

실행 후 xv6의 쉘에서 ppid 명령어를 실행하면 아래와 같이 잘 출력되는 것을 확인할 수 있었습니다.

```
exec getppid failed
$ ppid
My student ID is 2021026108
My pid is 4
My ppid is 2
$
```

Trouble Shooting

1. 만약 부모 프로세스가 없다면 어떻게 될 것이고 이를 어떻게 처리해야하는가?

찾아보니 xv6에서는 부모 프로세스가 없는 경우에 대해서 특별히 처리할 필요가 없습니다. 그 이유는 xv6에서 첫 번째 프로세스인 init 프로세스가 실행된 이후 모든 프로세스가 커널에 의해 생성되고 관리되기 때문에 부모 프로세스 없이 프로세스가 동작하는 경우는 무시해도 될 것이라고 생각했습니다.

2. 그렇다면 커널 이후로 가장 먼저 실행되는 프로그램인 init의 역할은 무엇일까?

init 프로세스가 xv6를 실행하면 가장 먼저 실행되며

`init: starting sh`이라는 문구를 출력하고 시스템이 종료될 때까지 계속해서 반복적으로 실행하는 프로세스로 다른 모든 프로세스의 상위 프로세스입니다.

```
// user/init.c의 반복구문
for(;;){
    // this call to wait() returns if the shell exits,
    // or if a parentless process exits.
    wpid = wait((int *) 0);
    if(wpid == pid){
        // the shell exited; restart it.
        break;
    } else if(wpid < 0){
        printf("init: wait returned an error\n");
        exit(1);
    } else {
        // it was a parentless process; do nothing.
    }
}
```